StripCheck: A Verifier-Gated Agent for Checking Medicine Packs Against India's Drug Quality Alerts

Aryanraj S Singh (1BM23CD008), Richik Chaudhuri (1BM23CD050),
Suraj B (1BM23CD061, team lead), Sushanth Deo (1BM23CD062)
Department of Computer Science and Engineering (Data Science), BMS College of Engineering (BMSCE), Bengaluru
B.E. CSE (Data Science), Semester VII
Agentic AI (26CD7PEAAI) mini-project report. Instructor: Dr. Kalyan Nagaraj

***Abstract*—India's Central Drugs Standard Control Organisation (CDSCO) publishes monthly lists of drug samples that failed quality tests or were declared spurious. The lists name individual batches, yet a person holding a strip of tablets has no practical way to check them. StripCheck is an offline agent that reads a medicine pack, from a photo or typed details, and decides whether its batch appears on these lists. Two properties of the problem shape the design. Optical character recognition (OCR) misreads single characters with high confidence, so a batch number read from foil cannot be trusted alone. Batch numbers also collide across unrelated products, and sibling batches of one product often differ by a single character, so a matching batch number is weak evidence on its own. StripCheck retrieves candidates by exact and confusion-aware batch search, corroborates each candidate field by field in the manner of record linkage, asks the person to confirm a doubtful character or the batch itself before it raises any alarm, and sends every draft verdict through a rule-based verifier that can abstain. Every loop is bounded and every model call has a deterministic fallback, so a check completes with no language model and no network. The alert index is built from CDSCO's online portal: 7,005 published rows from January 2019 to September 2026, of which 6,923 remain as indexed alerts after cleaning, de-duplication and quarantine. In these data the median alert appears 10 months after manufacture, when 63.5% of the flagged batches still have at least a year of shelf life left. On 60 rendered evaluation packs, the lookup baseline raised a false alarm on 8/8 collision packs and the agent on 0/8, with hit recall of 0.80 and 1.00 when a person answers the agent's questions. The preregistered decision rule gives: adopt. When nobody answers, the agent abstains on 36/60 packs and its hit recall drops to 0.20. A first run on the same packs gave the agent a hit recall of 0.82 and the same decision; three code defects found in that run were fixed before the reported run, so the reported numbers are optimistic.**

***Index Terms*—substandard medicines, LLM agents, human-in-the-loop, OCR post-correction, record linkage, abstention**

I. Problem and motivation

A. Niche area

Substandard and falsified medicines harm patients in every region, and low- and middle-income countries carry most of the burden [1], [2]. In India, central and state laboratories test drug samples taken from the market, and CDSCO publishes the batches that fail as monthly alerts of drugs Not of Standard Quality (NSQ) and of spurious drugs [3]. The volume of these alerts has grown quickly: the portal lists 404 entries for 2019 and 1,898 for 2025, with 1,890 more in the first nine months of 2026. CDSCO's recall guidelines carry a recall to consumers only for the most serious class of defects [4], so for most flagged batches the published list is the only signal a patient can reach.

TABLE I
Verdicts and the condition for each

Verdict	Returned when
FLAGGED	The confirmed batch matches an NSQ entry and the product plus dates or maker corroborate it.
SUSPECTED_FAKE	The confirmed batch matches an entry listed as purported spurious.
NOT_LISTED	No candidate survives exact and confusion-aware search of the lists loaded. This does not certify the medicine.
CANT_VERIFY	The evidence stays too weak after bounded steps and at most two questions; the agent abstains.

B. Specific problem

Nothing connects the list to the pack. The alerts are tables of product names, batch numbers, dates and manufacturers; the pack in a patient's or chemist's hand says which batch they hold. Someone who wants to know whether a strip is affected has to find the right month, read the batch number from the foil and compare it by hand. Packaging codes do not close this gap. Since August 2023 the top 300 brands carry a QR code under Schedule H2 [5], and such codes show where a pack came from, but WHO notes that verifying a unique identifier cannot check the medicine inside the package [6]. A batch that a licensed manufacturer made badly carries valid codes. Only the laboratory result says that the batch failed, and the alert list is where that result is published.

C. Decision the agent supports

StripCheck answers one question: *is this exact pack from a batch on a CDSCO alert list?* It returns one of four verdicts (Table I). Every alarm cites the list entry it rests on, and every result is advisory: a flag means "show this pack to your pharmacist", never "stop your medicine". Only the listed batch is flagged, never a brand.

D. Why a lookup script is not enough

Automating the check looks like a lookup, and a lookup script fails in two ways, both measured in the CDSCO data (Section I-E).

Reading. OCR on blister foil makes confident single-character errors. In our tests a scuffed `B` was read as `3` with a reported confidence of 1.0, and an enhanced re-read returned the same answer. A misread batch number either misses a listed batch or lands on another one.

Matching. Batch numbers are short and are chosen by each manufacturer, so the same string appears on unrelated products: 36 batch keys in the index are shared by products with no name token in common. Worse, 58 keys have a listed neighbour from the same manufacturer that differs in one easily confused character, for example `R21119` and `R21719`. A confident misread can therefore land exactly on a listed sibling, and every other field on the pack (product, maker, dates) will agree with it. Strips also often name the marketer while the list names the manufacturer, and spurious entries list the maker as "under investigation", so the maker field cannot always be compared.

The next step therefore depends on what the last tool found: an empty exact search calls for a variant search, a partial product match calls for a question or adjudication, and weak evidence calls for abstention. This is why the task suits an agent with a planning loop, a critic and a person in the loop.

E. Data observations that shape the design

CDSCO moved its alerts from monthly PDF tables to an online viewer in August 2025 [7], [8]. The viewer is embedded from CDSCO's SUGAM portal, whose page script requests rows for a chosen month from plain JSON endpoints that serve every month back to January 2019 [3]. Our snapshot of 25 October 2026 holds 7,005 rows: 6,956 NSQ rows over 93 months (January 2019 to September 2026) and 49 rows from the spurious-drug feed. A month has between 8 and 330 rows (median 48); state laboratories reported 3,822 rows and central laboratories 3,178.

The feed needs cleaning before it can be matched against a pack. Responses are encoded as ISO-8859-1, a brand name is present in only 13.8% of NSQ rows, some batch cells list several batches or carry a GS1 prefix, and 1.6% of rows lack a manufacturing date. The index has 6,667 distinct normalised batch keys, of which 268 appear more than once and 2.4% have fewer than four characters. 95 keys (1.42%) have another listed key one confusable substitution away, and 58 (0.87%) share the manufacturer. Manufacturers number batches in sequence and the regulator tests only samples, so an unlisted sibling of a listed batch is the ordinary case in a pharmacy, and these within-index counts are a lower bound.

Flagged batches are also still on sale when the alert appears. The median time from manufacture to the alert month is 10 months (interquartile range 6 to 15; 90th percentile 20). At the alert, the median batch has 14 months of shelf life left, 63.5% have at least twelve months left, and only 0.6% have expired. This motivates both a point-of-use check and re-checking the medicines a person has already saved.

F. Contributions

- An agent that combines confusion-aware retrieval, record-linkage grading, a confirmation gate before alarms and a rule-based verifier with abstention, and that addresses the sibling-misread false alarm a lookup cannot detect.
- A provenance-keeping ingest of CDSCO's portal with validation and quarantine, and an analysis of 93 months of alert data that measures batch collisions, confusable siblings and alert lag.
- An evaluation against a lookup baseline with preregistered decision rules, and an offline implementation with a web client, a native iOS client and a local API.

II. Related work and existing practice

A. Existing practice

WHO estimates that 10.5% of medical products in low- and middle-income countries fail quality tests [1], and a later review found that most failures are substandard rather than falsified [9]. A substandard batch from a licensed maker looks correct in every respect, so packaging cues do not help. In India, laboratories tested 116,323 samples in 2024–25 and found 3,104 not of standard quality and 245 spurious or adulterated [10]. Until mid-2025 a person who wanted to check a pack had to open the monthly PDF lists and search them by hand; since August 2025 the lists sit in a month-by-month web viewer [7]. Schedule H2 QR codes are assigned by each manufacturer and are mandatory for 300 brands [5]. They confirm the code and carry no test result, and rogue verification services have answered every query positively [6]. Field screening devices cost from US$3 to over US$60,000 and few have field evidence [11], [12], and a review of 25 medicine-verification apps found none that checked a regulator's quality list [13].

Several open tools built on CDSCO's data appeared in 2025 and 2026. Blacklisted offers typed batch lookup with an email digest [14]. Nazar reads QR codes and the printed batch line from a photo and returns "can't tell" when it sees look-alike characters [15]. SafeDose marks a batch hit with a different drug name as "caution" [16], and RecallOps India matches alerts against distributor stock with a human review queue [17]. Among the tools we found (searched October 2026), none asks the person to resolve a doubtful character during a check, corroborates product, brand, dates and maker on a consumer pack, or runs fully offline. StripCheck adds these, together with a verifier agent.

B. Reading codes on packs

On 27 pharmaceutical pack images, a region-based detector reached an F1 score of 80.9 against 47.1 for Tesseract [18]. StripCheck uses the PP-OCR model family [19], [20] through RapidOCR [21]. Confidence scores of modern networks are poorly calibrated [22], and on degraded text vision-language models fall back on language priors [23], so a fluent reading

of a code is no evidence that the code is right. Contrast enhancement did not improve licence-plate reading [24], consistent with our observation that an enhanced re-read repeats a confident misread.

C. Post-OCR correction and hotlist matching

Noisy-channel models rank candidate corrections by how likely the observed characters are given each candidate [25], [26], and a channel model over a closed vocabulary cut OCR word error by 70% [27]. Routing only disputed words to people raised accuracy to 99.1% [28], the pattern behind StripCheck's character questions. Licence-plate recognition is the closest operational analogue: its confusable pairs (B/8, D/0, I/1, S/5, Z/2) match those on batch codes [29], about one hotlist hit in five was a misread in one city audit [30], and a misread of 3 as 7 led to a stop at gunpoint, after which the court stressed visual verification before acting [31]. StripCheck's confirmation gate applies the same rule to medicine alarms.

D. Record linkage

Fellegi and Sunter sort record pairs into links, non-links and possible links that need manual review [32]. StripCheck maps these zones to an alarm, NOT_LISTED and a question or abstention. Errors in a blocking key lose true matches, so practical systems use several blocking passes [33], [34]; confusion variants of the batch key play that role, and their candidate recall can be reported separately from final accuracy [35]. Language models match entities nearly as well as fine-tuned matchers but are sensitive to the prompt [36], which supports a deterministic matcher. Indian brand names collide across unrelated molecules [37], so a brand match alone never raises an alarm.

E. LLM agents and external verification

Agents that interleave reasoning and tool use [38], [39] and that critique their own output [40], [41] are common, but models cannot reliably correct their own reasoning without external feedback [42], and planning works best when an external verifier checks the model's proposals [43]. Agents completed under half of the tasks in τ-bench [44], practitioner guidance favours simple, composable workflows over complex frameworks [45], and agent comparisons should report cost against simple baselines [46]. Small models suit narrow agent tasks [47]. StripCheck therefore lets a small model choose only among allowed actions, keeps a deterministic planner as fallback, and checks every draft with hard rules.

F. Abstention and risk communication

The reject option [48] and selective classification [49] formalise answering only when confident. Language models rarely abstain on medical questions [50], so StripCheck's abstention and wording come from rules and templates. People over-rely on automated advice [51], and cognitive forcing functions reduce this [52], which is why the confirmation asks the person to type specific characters from the pack. Adding a residual-risk sentence to a negative test result raised correct understanding from 54% to 89% [53], which supports the wording of NOT_LISTED. Consumer recall lookups elsewhere state how current they are [54], a convention StripCheck adopts.

TABLE II
COMPONENTS AND HOW THEY ARE BUILT

Component	Role	Built with
Supervisor	Runs sync, ingest and the cabinet re-check; deduplicates notifications	Python, SQLite
Ingest	Turns portal snapshots into a validated index; quarantines bad rows with reasons	Python, `urllib`, SQLite
Investigator	Bounded plan–act–observe loop over perception, retrieval and grading tools; asks the person	Python, rapidfuzz
Verifier	Checks every draft against hard rules; replaces a failing draft with CANT_VERIFY	Python rules
Perception	Reads QR codes and printed text, keeps provenance and character crops	RapidOCR, Apple Vision, Tesseract, zxing-cpp, OpenCV, Pillow
Local LLM	Optional planner choice and structured calls, never user-facing text	Ollama [55]
Interfaces	Streams steps and questions; shows cited verdicts	FastAPI, React, SwiftUI

III. AGENT DESIGN AND ARCHITECTURE

StripCheck has two kinds of work on two clocks. Background work runs when CDSCO publishes new lists: a supervisor fetches the portal, ingests the rows into a validated index and re-checks the medicines a person has saved. A check runs once per pack while the person waits: the investigator reads the pack, retrieves and grades candidates, asks the person when the evidence is missing and drafts a verdict, and the verifier checks the draft before anyone sees it. Figure 1 shows both lanes and Table II lists the components.

A. Investigator loop

The investigator is a plan–act–observe loop (Figure 2). The planner chooses the next action from those allowed in the current state; each action is a tool call whose observation is appended to the check's history, and control returns to the planner. In the default *rules* mode the planner follows a fixed priority order: a missing batch leads to a question, a batch leads to exact and then variant search, the candidates are graded, and the grades decide between a question, a draft and abstention. In *hybrid* mode a local model may choose at branch points, but only from the actions the rules allow; a timeout, a schema failure or a disallowed choice falls back to the rule choice, and the history records which planner decided.

A typical misread check runs as follows. OCR reads the batch as `LM24088`, and exact search finds nothing. Variant search reaches the listed key `LM2408B` by one substitution (8 for B). Grading finds that product, dates and maker agree, but the batch came from a substitution, so the planner asks the person to confirm the doubtful character, shown with the camera's crop. The person answers `B`, the confirmed key is searched exactly, and the draft FLAGGED passes the verifier. A

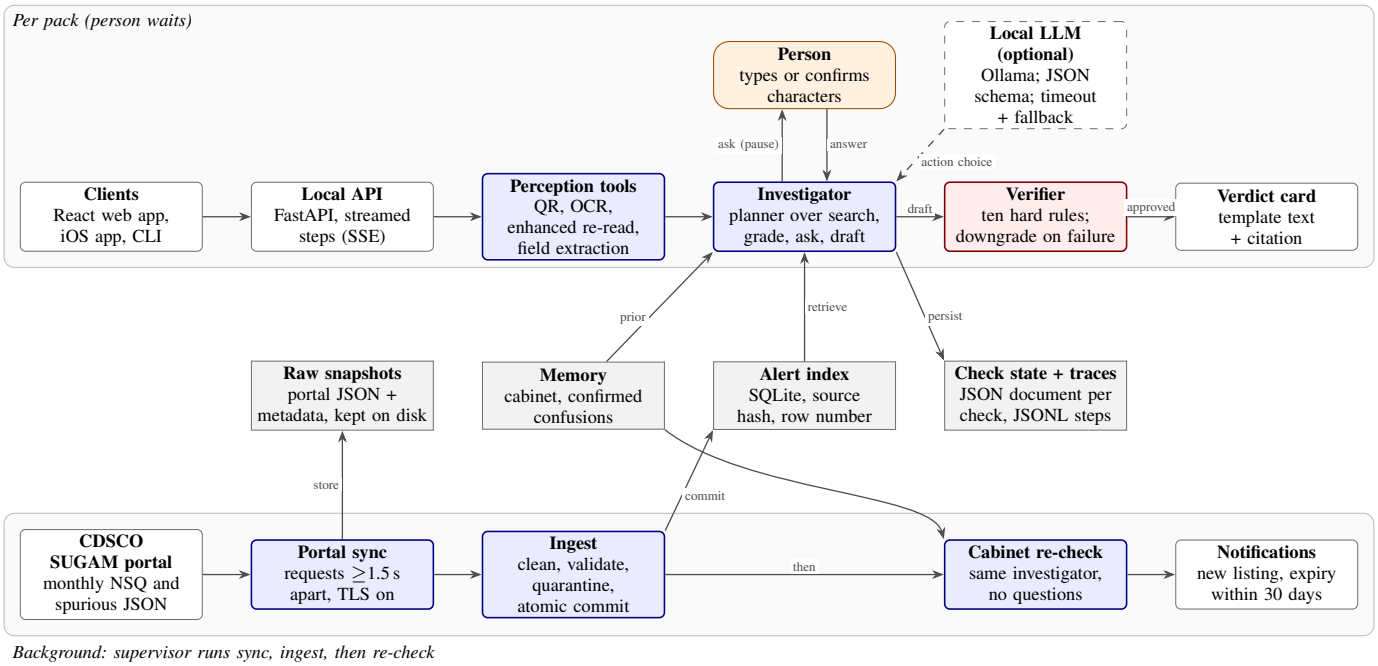


Fig. 1. StripCheck architecture as built. The per-pack lane runs while the person waits; the background lane runs after each sync. Both read the same alert index. Blue boxes are agents or agent tools, the red box is the verifier gate, grey boxes are local stores, and the dashed box is optional.

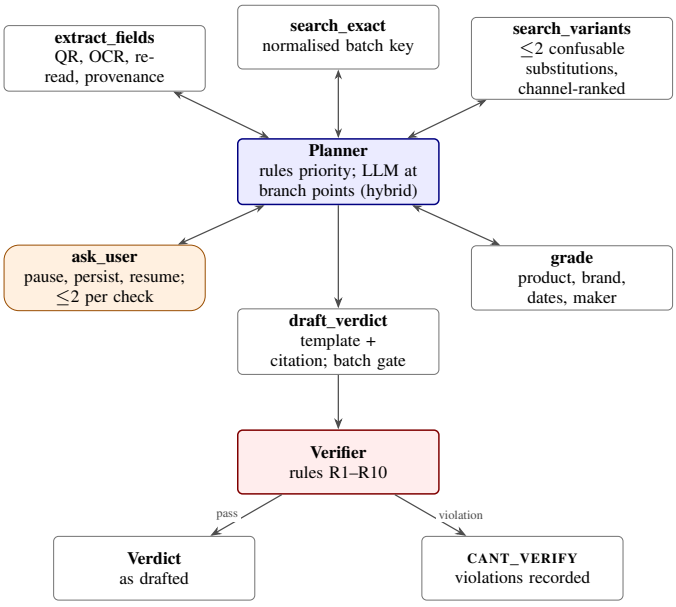


Fig. 2. Investigator graph. Double-headed arrows are tool calls that return to the planner. The only exit is through the verifier, which approves the draft or downgrades it to CANT_VERIFY.

lookup script would have answered NOT_LISTED (Section IV shows the recorded trace).

B. Confirmation gate

Before any alarm the batch must come from a QR code, from the person (typed or confirmed), or from two independent reads that agree. Otherwise the investigator asks the person to type every look-alike character of the batch, in the order of doubt that the confusion prior gives. It does not offer a yes/no button, because people habituate to such confirmations and tap without looking. This gate targets the sibling false alarm of Section I-E, where every other field agrees and only a person looking at the pack can tell the batches apart.

TABLE III
BOUNDS ENFORCED IN CODE

Bound	Value
Planner steps per check (verifier rule R6)	16
Questions to the person per check	2
Substitutions per searched variant	2
Variant keys looked up per search	2,000
Candidates graded per check	20
Language-model attempts per call (each with a timeout)	2
Portal request spacing	≥1.5 s
Longest image side passed to a reader	2,400 px

C. Bounds

Every loop is bounded (Table III), and every model, reader and network call has a timeout and a fallback, so a check finishes with the language model, the tracer and the network all switched off.

IV. AGENTIC CONCEPTS DEMONSTRATED

Table IV maps each concept required by the course, and each bonus concept, to the part of StripCheck that implements it and the evidence a reviewer can inspect. Each pattern fixes a specific way in which the lookup script of Section I fails.

TABLE IV
AGENTIC CONCEPTS, WHERE THEY LIVE AND THE EVIDENCE FOR EACH

Concept	Tier	Where in StripCheck	Evidence
Tool use	Required	Perception (`decode_qr`, `ocr_read`, `extract_fields`), retrieval (`search_exact`, `search_variants`), `grade`, `ask_user`, `draft_verdict`; ingest tools	Every call is a step in the check's history with its action, planner source, summary and duration (Table V)
Agentic RAG	Required	Exact batch, then confusion variants; each candidate graded on batch, product, brand, dates and maker; re-query after an answer; abstain when grades stay weak	Steps 2–4 and 7–9 of Table V; candidate grades in the check record
Planning and reasoning	Required	Planner loop with rule priority; hybrid mode lets a local model choose among allowed actions at branch points	The *source* field of each step (rules, LLM, or rules after a fallback)
Observability	Required	JSON-lines trace per check, streamed step timeline in the interface	`data/traces/<check>.jsonl`; live timeline in the web and iOS clients
Evaluation	Required	Synthetic diagnostic scenarios, labelled evaluation set, lookup baseline	Sections VI and VII
Multi-agent	Bonus	Supervisor, ingest, investigator and verifier with separate jobs and clocks	Cabinet re-check report and notifications; verifier rule results
Memory	Bonus	Saved cabinet re-checked after each sync; confirmed character corrections stored as a confusion prior	Trace line "Prior confirmed corrections: 8 to B: 1. Confirmation is still required."
Human in the loop and guardrails	Bonus	Five question kinds; template-only verdict text; banned phrases; confirmation gate; verifier	Question payload and the resumed answer; downgrade with rule identifiers
Durable workflow	Bonus	Whole check state is a JSON document saved in SQLite after every run; a question can wait across restarts	Resume endpoint continues a stored check; repeated import adds no rows
Experimentation	Bonus	Lookup baseline against the agent, with and without a person answering	Section VII

TABLE V
RECORDED STEPS FOR THE MISREAD SCENARIO (RULES PLANNER)

#	Action	Observation
1	`extract_fields`	Batch read as `LM24088` from the pack
2	`search_exact`	0 candidates
3	`search_variants`	1 candidate (`LM2408B`, one substitution); no printed character changed
4	`grade`	Product, dates and maker agree; batch is a variant, so status is *needs character confirmation*
5	`ask_user`	Confirm character 7: printed `8` or `B`? (check paused and stored)
6	`answer`	The person chooses `B`; batch evidence becomes user-confirmed
7	`search_exact`	1 candidate
8	`search_variants`	0 candidates
9	`grade`	All fields agree; status *accept*
10	`draft_verdict`	FLAGGED with citation; verifier passes

TABLE VI
VERIFIER RULES

Rule	Requirement
R1	The label is one of the four verdicts
R2	An alarm needs an accepted candidate, an exact or confirmed batch, gate evidence, a product match (or a partial match when dates and maker both match) and no field mismatch
R3	SUSPECTED_FAKE only for spurious entries, FLAGGED only for NSQ-type entries
R4	NOT_LISTED needs a grounded batch, both searches, no pending or accepted candidate, loaded data and no truncation
R5	An alarm cites a traceable source; no text contains a banned phrase
R6	At most 16 steps and 2 questions
R7	No unresolved date conflict outside CANT_VERIFY
R8	Every field used has grounded provenance (QR, OCR, carton photo or the person)
R9	NOT_LISTED states the lists loaded and the latest month
R10	Only reviewed languages are shown

A. A recorded trace

Table V is the step history that the engine recorded for the synthetic misread scenario of Section III, with the summaries shortened. The planner suspects a misread only because the exact search came back empty, and the second search pass runs only because the person changed the batch. A fixed pipeline cannot produce this sequence.

B. Collision and sibling traps

In the collision scenario the pack carries a listed batch number on a different product. Exact search returns the entry, grading marks the product as a mismatch and rejects the candidate, and the verdict is NOT_LISTED in five steps. A lookup script raises an alarm here. In the sibling scenario every field of the pack agrees with a listed entry, so grading accepts it, but the batch came from OCR. The gate asks the person to type the doubtful characters; the person types `LM2408R`, the agent rejects the listed key, searches again and returns NOT_LISTED.

C. The verifier as critic

The verifier is independent of the drafting code: it re-checks the draft against the evidence in the stored state. Table VI lists its rules. To show the gate at work we passed it a FLAGGED draft for the clean scenario whose batch came from OCR alone. It returned CANT_VERIFY with the violations *R2: corroborated, confirmed batch evidence is required* and *R8: every used field needs grounded provenance*. On any violation the verifier replaces the draft with the CANT_VERIFY template and stores the rule identifiers, so an unsupported alarm cannot reach the person.

D. Feedback loop and memory

When a person corrects a character on a photographed pack, the server stores the pair (read character, printed character) in a confusion table, once per answer. On the next check the prior reorders the variant candidates by the channel score of Section V and orders the characters the confirmation asks about, and the trace says so (Table IV, memory row). The

question itself adds, from a template: "Earlier checks on this computer confirmed that the photo reader read B as 8 once. Please still check the printed character yourself." The prior never skips the question, because two batches of one product from one maker can differ by exactly the character in doubt. The cabinet is the second memory: after each sync the supervisor runs every saved pack through the same investigator without questions and creates a notification when one now matches a listed entry, or when its printed expiry month is less than 30 days away.

V. Implementation

A. Stack

The engine is plain Python 3.12 in one module per concern (`core/`), with SQLite for every store. A FastAPI server (`server/`) exposes checks, answers, alerts, sources, quarantine, the cabinet, notifications, diagnostics and traces, and streams each step to the client as server-sent events. The web client is React with TypeScript, built with Vite, and a phone on the same network can use it after a token-protected pairing. A native SwiftUI client for iOS uses the same API, and a command-line interface runs sync, checks, diagnostics and the evaluation. No paid service is used, and no model is downloaded automatically. The optional language model runs locally under Ollama with JSON-schema outputs [55].

B. Data ingest

The sync tool discovers the published months for the NSQ and spurious tabs of the portal, fetches each month with requests spaced at least 1.5 s apart and with TLS verification on, and stores the exact response bytes with a metadata file and a SHA-256 hash. A normal sync refreshes the newest three months, because late state reports change them; a full sync imports the history. Ingest then decodes each snapshot as Latin-1, repairs only reversible encoding damage in descriptive fields, splits multi-batch cells and maker–marketer cells, parses dates to year and month, and normalises the batch into a match key. Each row gets flags (missing date, manufacture after expiry, non-drug product and others) and, when it cannot be matched safely, quarantine reasons such as a missing product, a placeholder batch or a batch cell that holds a date. Quarantined rows stay visible in the interface. Imports are atomic and idempotent: repeating an unchanged import adds no rows, and every alert row keeps its source URL, hash and row number for the citation. An offline sync rebuilds the index from the stored snapshots alone.

C. Perception

A photo is validated and resized so that no reader sees more than 2,400 pixels on a side. Barcodes are read first (zxing-cpp, then Apple Vision on macOS, then OpenCV's QR detector), in a separate process with a deadline, because a decoded GS1 payload is error-checked. Text is read by RapidOCR with PP-OCR models [19], [21], by Apple Vision on macOS or by Tesseract, whichever is installed. If no batch is found, the image is read once more after grey-scaling, auto-contrast and sharpening. Field extraction uses label-anchored patterns and keeps the provenance of every value. When the QR and the printed batch disagree, the printed read wins and the batch loses its confirmed status. A failed read never creates pack details; the person is asked to type them.

D. Retrieval with an OCR channel model

Retrieval uses the normalised batch key as a blocking key. Exact lookup comes first, and confusion variants follow: the keys reachable from the read key r by at most two substitutions from a table of characters that OCR confuses on printed codes (for example `B`/`3`/`8`, `0`/`O`/`D`, `1`/`7`/`I`). Variants are ordered by the number of substitutions and then by a channel score

$$s(v \mid r) = \sum_{i \in S} \log \frac{n_{r_i v_i} + \alpha}{n_{r_i r_i} + \alpha}, \tag{1}$$

which follows the noisy-channel view of spelling and OCR correction [25]–[27]. Here S is the set of substituted positions, n_{ab} counts how often a read a was confirmed as printed b, and $\alpha = 1$ smooths the counts. The counts grow with every character a person confirms. The score only orders candidates and chooses which characters to ask about; it never replaces the question. In the server, retrieval first fetches only the exact and confusable keys through an index on the active batch keys, so a check does not load the whole table.

E. Grading as record linkage

Each candidate is compared with the pack field by field. Product evidence is the share of the entry's name tokens found on the pack (match from 0.8, partial from 0.5), brand evidence is the same measure on the brand, dates are compared by year and month, and the maker is compared with a token-set similarity of at least 85 after the names are normalised. A product or brand mismatch rejects the candidate as a collision. An exact or confirmed batch whose product is unknown, or whose maker disagrees, needs more evidence, so the agent asks for a carton photo or typed details. A one-digit date difference on an exact batch becomes a question, because one misread digit explains it; a larger difference becomes a conflict that ends in CANT_VERIFY instead of a silent NOT_LISTED. A partial product match without agreeing dates and maker goes to adjudication. A variant batch needs a character confirmation. Spurious entries skip the maker check, since their listed maker is often "under investigation.

F. Questions and durable state

The investigator asks five kinds of question: type the batch, confirm the batch (type every look-alike character, the most doubtful first), confirm a character (with the camera's crop of the glyph), confirm a printed date, and add a carton photo or typed details. Every answer can be "can't tell", which records a decline. The whole state of a check (fields, provenance, history, candidates, the pending question and the evidence level of the batch) is JSON and is saved as a document in SQLite after every run. Answering a question calls the resume endpoint, which reloads the stored state and continues from it,

TABLE VII
DEVIATIONS OF THE BUILT SYSTEM FROM THE REVIEW 2 DESIGN

Review 2 design	As built, and why
LangGraph graphs with SQLite checkpoints	A bounded custom engine whose whole state is a JSON document in SQLite; simpler to test and to resume from any client (D18)
Arize Phoenix traces	JSON-lines trace per check and a streamed timeline; OpenTelemetry export to Phoenix is opt-in (D18)
Streamlit app	FastAPI with a React web client, plus a native iOS client; streaming questions fit poorly with Streamlit's rerun model (D2, D16)
Monthly PDF parsing first	Portal JSON is the end-to-end source; PDF helpers exist but the full PDF repair pipeline is future work (D1, D18)
Vision-language second reader	Not shipped; a QR code, or the same batch read from two photos, counts as independent evidence
Verifier sends a draft back once	Send-back once only when the language-model planner drafts before the variant search; any other failing draft is replaced by CANT_VERIFY with the violated rules recorded
Engine as first evaluated	Three defects fixed after the first evaluation run (D25): grading sees the full product text, R2 accepts the partial match that grading accepts, and the batch confirmation asks every look-alike position (Section VII-C)

so a question can wait across restarts. A check that is already running, or that changed since the client loaded it, is refused, which prevents two answers from racing.

G. Verdict text and safety

All user-facing text comes from one templates module. Models never write verdicts. The templates state the list, month, failed test and reporting laboratory for an alarm, and the lists loaded and the latest month for NOT_LISTED, together with a sentence that the result does not certify the medicine. A regular expression of banned phrases ("safe", "genuine", "authentic", "fake medicine", "stop taking" and their Hindi equivalents) is enforced by the verifier and by tests. The interface never shows NOT_LISTED in green. Demonstration records are marked synthetic and are excluded from real checks.

H. Deviations from the design

The implemented system differs from the design presented at Review 2 in the places listed in Table VII; each change is recorded in the project's decision log.

VI. OBSERVABILITY AND EVALUATION

A. Tracing

Every check writes a JSON-lines trace under `data/traces/`, one file per check. Each line carries a timestamp, the check identifier, the event (step, question, answer, learning, verdict or error), the action, the planner source (rules, LLM, or rules after a fallback), a one-line summary, the duration and, for model calls, the model name, token counts and whether a fallback occurred. The same events stream to the client, which draws them as a live timeline next to the batch, shown as a strip of character cells that turn from *read* to *uncertain* or *confirmed*. Writing a trace can never fail a check: errors in the tracer are swallowed, and the file is bounded when it is read back. Spans can additionally be exported through OpenTelemetry to a local Arize Phoenix server using the OpenInference conventions [56], [57]; the export is switched on by a setting, probes the collector with a short timeout, and a check runs the same without it. Each check becomes one trace: an agent span for the check, a chain span per planner decision with its tool or model span beneath, and a guardrail span for the verifier.

B. Test sets

Synthetic diagnostics. Eleven authored scenarios, each with an expected verdict, cover a clean listed batch, a batch absent from the index, a one-character misread, a spurious entry, a missing carton, a batch collision, missing product text, an unreadable batch, a date disagreement, a neighbouring sibling batch and an unreadable pack. They run offline in the test suite and from the interface, and they check behaviour, not accuracy on real medicines.

Evaluation set. 60 cartons are rendered from real alert rows in the local index, each marked as a demonstration prop, with a fixed scenario mix: 8 clean, 8 with a QR code, 8 with glare, 8 misread, 8 spurious, 8 collision, 8 unlisted and 4 sibling packs. Thresholds were tuned on a development set (seed 1) only, and results are reported on a separate evaluation set (seed 7). Collision packs carry a listed batch number on a different product and maker, which models the reuse of short codes seen in the data. Misread and sibling packs print the misread string on the carton, so the OCR error is simulated and the oracle answers from the batch the pack was meant to carry. The evaluation seed was run twice, with three code fixes in between; Section VII-C reports both runs.

C. Systems compared

The baseline is a lookup script: QR code or OCR, a regular expression for the batch, exact lookup, and any hit reported as an alarm. It is compared with the agent using the rule planner and the agent using the hybrid planner. Each arm runs twice: with an oracle user that answers every question truthfully from the ground truth, and with every question skipped, which shows what the questions buy.

D. Metrics and statistics

Hit recall is the share of packs whose truth is FLAGGED or SUSPECTED_FAKE that receive the correct label. Type accuracy is the share of detected positives with the right one of the two labels. The false-alarm rate is the share of packs whose truth is NOT_LISTED that receive an alarm, reported overall and on collision packs alone. We also report the abstention rate, questions per pack, median and 95th-percentile latency, and safety violations (banned phrases or alarms without a citation, required to be zero). Agent comparisons without cost are misleading [46], so latency and questions are reported for every arm. Rates carry 95% Wilson intervals [58], [59], and paired comparisons on the same packs use the exact McNemar test [60], [61]. With at most 8 packs per scenario a difference of one or two packs is noise, so per-scenario results are given as counts.

TABLE VIII
HEADLINE METRICS ON THE EVAL SET (SEED 7, $n = 60$), HITL: ORACLE USER. RATES AS VALUE (k/n; WILSON 95% INTERVAL).

Metric	Baseline (lookup)	Agent (rules)	Agent (hybrid)
Hit recall	0.80 (32/40; 0.65-0.90)	1.00 (40/40; 0.91-1.00)	1.00 (40/40; 0.91-1.00)
Hit recall (clean + QR)	1.00 (16/16; 0.81-1.00)	1.00 (16/16; 0.81-1.00)	1.00 (16/16; 0.81-1.00)
Type accuracy	1.00 (32/32; 0.89-1.00)	1.00 (40/40; 0.91-1.00)	1.00 (40/40; 0.91-1.00)
False-alarm rate (all NOT_LISTED)	0.60 (12/20; 0.39-0.78)	0.00 (0/20; 0.00-0.16)	0.00 (0/20; 0.00-0.16)
False-alarm rate (collisions)	1.00 (8/8; 0.68-1.00)	0.00 (0/8; 0.00-0.32)	0.00 (0/8; 0.00-0.32)
False-alarm rate (siblings)	1.00 (4/4; 0.51-1.00)	0.00 (0/4; 0.00-0.49)	0.00 (0/4; 0.00-0.49)
Abstention rate	0.00 (0/60; 0.00-0.06)	0.00 (0/60; 0.00-0.06)	0.00 (0/60; 0.00-0.06)
Overall accuracy	0.67 (40/60; 0.54-0.77)	1.00 (60/60; 0.94-1.00)	1.00 (60/60; 0.94-1.00)
Questions per pack	0.00	0.60	0.60
Steps per pack	1.0	8.0	8.0
Latency median / mean / p95 (s)	1.33 / 1.36 / 1.82	1.39 / 1.43 / 1.86	1.51 / 3.28 / 8.96
Reasoning only, median / p95 (s)	0.000 / 0.000	0.061 / 0.103	0.099 / 7.672
Safety violations	0	0	0
LLM steps / fallback steps	0 / 0	0 / 0	28 / 0

TABLE IX
HEADLINE METRICS ON THE EVAL SET (SEED 7, $n = 60$), HITL: EVERY QUESTION SKIPPED. RATES AS VALUE (k/n; WILSON 95% INTERVAL).

Metric	Baseline (lookup)	Agent (rules)	Agent (hybrid)
Hit recall	0.80 (32/40; 0.65-0.90)	0.20 (8/40; 0.10-0.35)	0.20 (8/40; 0.10-0.35)
Hit recall (clean + QR)	1.00 (16/16; 0.81-1.00)	0.50 (8/16; 0.28-0.72)	0.50 (8/16; 0.28-0.72)
Type accuracy	1.00 (32/32; 0.89-1.00)	1.00 (8/8; 0.68-1.00)	1.00 (8/8; 0.68-1.00)
False-alarm rate (all NOT_LISTED)	0.60 (12/20; 0.39-0.78)	0.00 (0/20; 0.00-0.16)	0.00 (0/20; 0.00-0.16)
False-alarm rate (collisions)	1.00 (8/8; 0.68-1.00)	0.00 (0/8; 0.00-0.32)	0.00 (0/8; 0.00-0.32)
False-alarm rate (siblings)	1.00 (4/4; 0.51-1.00)	0.00 (0/4; 0.00-0.49)	0.00 (0/4; 0.00-0.49)
Abstention rate	0.00 (0/60; 0.00-0.06)	0.60 (36/60; 0.47-0.71)	0.60 (36/60; 0.47-0.71)
Overall accuracy	0.67 (40/60; 0.54-0.77)	0.40 (24/60; 0.29-0.53)	0.40 (24/60; 0.29-0.53)
Questions per pack	0.00	0.60	0.60
Steps per pack	1.0	8.0	8.0
Latency median / mean / p95 (s)	1.33 / 1.36 / 1.82	1.41 / 1.45 / 1.87	1.51 / 3.44 / 12.64
Reasoning only, median / p95 (s)	0.000 / 0.000	0.095 / 0.131	0.099 / 11.352
Safety violations	0	0	0
LLM steps / fallback steps	0 / 0	0 / 0	32 / 0

E. Results

The tables in this section give the headline metrics with the oracle user, the same metrics with every question skipped and the per-scenario counts. They are from the second run, after the fixes of Section VII-C. Perception is shared by all arms and reads each image once; the batch read equalled the rendered text on 60/60 images, at a median of 1.33 s per image. All numbers are written by the evaluation command into `paper/generated/` and are not edited by hand.

VII. EXPERIMENTATION: A/B TEST AND DECISION

A. Design

The experiment asks whether the agentic parts are worth their cost. Arm A is the lookup script, arm B the agent with the rule planner and arm C the agent with the hybrid planner, all on the same evaluation packs, each with the oracle user and with questions skipped. Before the evaluation run we registered a prediction and a decision rule [62].

Prediction. On clean packs all arms tie. On collision packs the baseline raises a false alarm on every one, while the agent stays at or below 10%. On misread packs the baseline misses every true flag, while the agent with the oracle recovers at least 80% at no more than one question per pack. Arm C matches arm B on verdicts.

Decision rule R-A. Adopt the agent in place of the lookup script if its false-alarm rate on collision packs is at least 50% lower than the baseline's (relative) and its hit recall is no more than five points lower. A second rule, R-C, makes the hybrid planner the default only if its accuracy is not significantly lower than the rule planner's and its median latency is at most twice as high. We report the outcome whether or not the rules hold.

TABLE X
CORRECT VERDICTS PER SCENARIO (k/n), HITL: ORACLE.

Scenario	Baseline (lookup)	Agent (rules)	Agent (hybrid)
clean	8/8	8/8	8/8
glare	8/8	8/8	8/8
misread	0/8	8/8	8/8
qr	8/8	8/8	8/8
spurious	8/8	8/8	8/8
collision	0/8	8/8	8/8
genuine	8/8	8/8	8/8
sibling	0/4	4/4	4/4

TABLE XI
CORRECT VERDICTS PER SCENARIO (k/n), HITL: QUESTIONS SKIPPED.

Scenario	Baseline (lookup)	Agent (rules)	Agent (hybrid)
clean	8/8	0/8	0/8
glare	8/8	0/8	0/8
misread	0/8	0/8	0/8
qr	8/8	8/8	8/8
spurious	8/8	0/8	0/8
collision	0/8	8/8	8/8
genuine	8/8	8/8	8/8
sibling	0/4	0/4	0/4

B. Outcome

Table XIII collects the numbers rule R-A uses, on 60 packs of evaluation seed 7 checked against an index of 6,923 alerts. With the oracle user the decision under R-A is: **adopt**. The exact McNemar test on overall correctness gives $p =$ 2e-06 (discordant packs: 20 where only the agent is correct, 0 where only the baseline is). With every question skipped the decision is: do not adopt. The agent's hit recall then falls to 0.20 (8/40) and its abstention rate rises to 0.60, while its false-alarm rate stays at 0.00. When nobody confirms the batch, the agent abstains, so its recall depends on the person answering. The packs it still gets right without answers are those whose batch came from a QR code and the collision and unlisted packs (Table XI).

The registered prediction. Table XII gives each part with its counts. The clean-pack part held, the collision part held, the misread part held and the part on hybrid and rule labels held. A further check, whether the hybrid planner does better on partial product matches, was not evaluable, because no evaluation pack produced a partial product match. Hit recall on clean and QR packs was 16/16 for the baseline and 16/16 for the agent. The agent paid 0.60 questions per pack and a median latency of 1.39 s against 1.33 s for the baseline, most of which is OCR time in both arms. No arm produced a safety violation.

Rule R-C. The hybrid planner consulted the local model on 28 steps with 0 fallbacks to rules. It reached hit recall 1.00 and collision false alarms 0/8, and the prediction that it matches the rule planner on every label held. Its median latency was 1.51 s against 1.39 s for the rule planner. Identical labels leave no discordant pair, so its accuracy is not lower,

TABLE XII
REGISTERED PREDICTIONS AND OUTCOMES (HITL: ORACLE).

Prediction	Outcome	Counts
Clean packs: all arms tie	held	baseline 8/8; agent_rules 8/8; agent_hybrid 8/8
Collisions: baseline false-alarms on every one; agent at most 10%	held	baseline 8/8, agent_rules 0/8
Misreads: baseline misses every flag; agent with oracle recovers at least 80% at no more than 1 question per pack	held	baseline detected 0/8; agent_rules correct 8/8 at 1.0 questions per pack
Hybrid matches rules on verdicts	held	60/60 identical labels
Hybrid beats rules on partial product matches	not evaluable	no item produced a partial product match

TABLE XIII
INPUTS TO DECISION RULE R-A (ORACLE USER, k/n IN BRACKETS)

Metric	Lookup	Agent (rules)
Hit recall	0.80 (32/40)	1.00 (40/40)
False alarms, collisions	1.00 (8/8)	0.00 (0/8)
False alarms, siblings	1.00 (4/4)	0.00 (0/4)
False alarms, all unlisted	0.60 (12/20)	0.00 (0/20)
Overall accuracy	0.67 (40/60)	1.00 (60/60)
Questions per pack	0.00	0.60
Median latency (s)	1.33	1.39

and its median latency is within the factor of two that R-C allows. R-C therefore holds, and by the registered rule the hybrid planner may be the default wherever a local model runs. The median hides the cost in the tail. The 95th-percentile latency was 8.96 s against 1.86 s, because each model call takes seconds on a laptop CPU. On these packs the model changed no verdict; it adds a written reason for each branch decision to the trace.

C. A first run and three fixes

The evaluation seed was run twice, and the tables above are from the second run. The first run (commit `e283407`) used the same packs, arms and decision rules. It also gave **adopt** under R-A: with the oracle user the agent reached hit recall 0.82 (33/40) with 0/8 collision false alarms, and McNemar gave $p = 0.0106$ (18 packs where only the agent was correct, 5 where only the baseline was). Two parts of the prediction failed: the agent was correct on 7/8 clean packs and on 6/8 misread packs. Table XIV sets the two runs side by side.

The 7 alarms the agent missed in the first run were all on cartons that print a brand line above the generic name. Reading their traces, and reviewing the code for the project guide, showed three defects. (1) The investigator kept only the product and brand fields for grading, so a brand line could hide the generic name and a listed batch was rejected as a collision; grading now sees the whole product text the reader returned. (2) Verifier rule R2 demanded a full product match, while grading accepts a partial match when dates and maker both match, so a person could answer every question and still receive CANT_VERIFY; R2 now accepts what grading accepts.

TABLE XIV
AGENT WITH THE RULE PLANNER ON SEED 7 BEFORE AND AFTER THE D25 FIXES (ORACLE USER UNLESS STATED)

Metric	Run 1 (`e283407`)	Run 2 (reported)
Hit recall	33/40	40/40
Hit recall, clean and QR	12/16	16/16
False alarms, collisions	0/8	0/8
Overall accuracy	53/60	60/60
Questions per pack	0.55	0.60
McNemar p against lookup	0.0106	2e-06
Decision under R-A	adopt	adopt
Prediction: clean tie	did not hold	held
Prediction: misread recovery	did not hold	held
Hit recall, questions skipped	5/40	8/40
Abstentions, questions skipped	31/60	36/60

(3) The batch confirmation asked only the first two look-alike positions, so a misread at a third position could confirm the wrong batch; it now asks every look-alike position. The fixes are recorded as decision D25. Each one changes a general rule of the engine, and none refers to a pack, batch or threshold of the evaluation set. They were still made after we had seen the first run on the evaluation seed, so the second run is no longer a held-out test, and its numbers are optimistic. A fresh seed and real photos are needed for an unbiased estimate.

D. What the decision informed

The decision rule fixes the default configuration of the released system. Because R-A holds, the agent replaces the lookup in every client, and the lookup survives only as the baseline arm. The engine's built-in setting is the rule planner, which is also what runs on a laptop with no model installed; the hybrid planner is switched on per check or through a setting, and an unreachable model falls back to rules within one timeout. The language model therefore remains an optional component that the system never needs to reach a verdict.

VIII. LIMITATIONS, ETHICS AND SAFETY

A. Limitations

A NOT_LISTED verdict is only as good as the lists behind it. CDSCO tests samples, so most batches on the market are never tested, and an unlisted batch can still be substandard; StripCheck states this on every such result and shows how recent its lists are. Every pack in the evaluation and in the demonstrations is synthetic: we rendered the packs from alert rows and took no photos of real medicine strips. Rendered packs flatter OCR, and embossed foil, curved bottles and poor light produce confusions the table may not contain, so the results say nothing yet about accuracy on real strips. Labels are read in English only. The oracle user answers every question perfectly, while real people may misread the pack or skip the question. Each scenario has few packs, so small differences between arms are noise. Latency is measured on a laptop CPU. The index covers the portal from January 2019, so older alerts are absent, and legacy PDF months enter only through helpers that are not yet a complete pipeline. Regional-language templates await review by a native speaker and are not shown.

B. Threats to validity

Thresholds were tuned on the development seed only, and the evaluation seed was generated once, but both sets come from the same generator, so they share its blind spots. The evaluation seed was also run twice: three code defects found in the first run were fixed before the second (Section VII-C). The fixes are general, yet the reported run is no longer a held-out test, and its scores with the oracle user are an optimistic estimate. Collision and sibling traps are built from real batch keys, yet their frequency in the test set is far higher than on a person's shelf. At realistic base rates even a small false-alarm rate can produce more false alarms than true ones, so the positive predictive value of an alarm in use will be lower than the test-set rates suggest.

C. Ethics and safety

StripCheck gives advice and directs every result to a pharmacist. It never says that a medicine is safe or genuine, never tells anyone to stop a medicine and flags only the listed batch, never a brand. Verdict text comes from fixed templates, and the language model cannot write user-facing text. Text read from packs and alerts enters prompts only as data, the model may choose only among allowed actions, and its endpoint must be on the local machine, which limits what injected text could do. Photos, checks and alert data stay on the user's machine; no personal data is collected, and phone access requires a pairing token. CDSCO's website policy asks that its content be acknowledged, so the alert data is kept locally and cited on every alarm, and this report publishes only derived statistics. StripCheck is a student project and not a CDSCO service, and the interface says so. The code was written with AI coding assistants under the team's direction, and every module was reviewed and tested by the team.

IX. Conclusion and future work

StripCheck links CDSCO's batch-level quality alerts to the pack in a person's hand. Its design follows from two measured properties of the data: batch numbers collide across products, and listed batches have confusable siblings, so a batch match alone cannot justify an alarm and a confident OCR read cannot justify a miss. The agent searches for likely misreads, corroborates every field, asks the person to read the characters in doubt, and lets an independent verifier downgrade any draft the evidence does not support. Every loop is bounded and every model call has a fallback, so the whole check runs offline on a laptop. In the A/B test the agent removed the baseline's collision and sibling false alarms (0/8 and 0/4 against 8/8 and 4/4) and raised hit recall from 0.80 to 1.00 when the person answers its questions. The preregistered rule's decision was: adopt. A first run on the same packs reached the same decision with hit recall 0.82; the reported run follows three general fixes made after that run, so its numbers are optimistic. Without answers the agent abstains on 36/60 packs with a false-alarm rate of 0.00, which shows that the questions carry the recall. On whether the hybrid planner matches the rule planner on every label, the registered prediction held, and the hybrid paid for its model calls in tail latency.

Future work follows from the limitations. A complete adapter for legacy PDF months and for state-level feeds would extend coverage before 2019. Real photos of strips under varied light, and a usability study that tests whether people understand what NOT_LISTED means, would measure the gap that rendered packs hide. Reviewed Hindi and Kannada templates, voice prompts and pictograms would serve users who do not read English. A pharmacist workflow that checks a whole delivery against the lists would move the check to the point where most flagged batches can still be stopped.

Team contributions

Every member can explain the whole system; Table XV records who led each part. The code was written with AI coding assistants under the team's direction.

TABLE XV
Contribution table

Member	Owned work	Led review
Aryanraj S Singh (1BM23CD008)	Investigator and verifier, planner, tracing, interfaces	R3 trace walkthrough
Richik Chaudhuri (1BM23CD050)	Portal sync, ingest, alert index, matching, cabinet and memory	Data plan
Suraj B (1BM23CD061), lead	Pack-reading tools, demonstration packs, integration, data analysis, report	R1, demos
Sushanth Deo (1BM23CD062)	Evaluation harness, test sets, A/B experiment, results	R4 results

Acknowledgements and data

Alert data: Central Drugs Standard Control Organisation, Government of India [3], used with acknowledgement and kept locally. Software: PP-OCR models through RapidOCR [19], [21], Ollama [55], OpenTelemetry and OpenInference [56], [57], FastAPI, React, SQLite, OpenCV, Pillow, rapidfuzz and zxing-cpp.

References

[1] World Health Organization, "1 in 10 medical products in developing countries is substandard or falsified," WHO News Release, Nov. 2017, 28 Nov 2017; accompanies the WHO GSMS report and the public-health and socioeconomic impact study. Accessed 25 Oct 2026. [Online]. Available: https://www.who.int/news/item/28-11-2017-1-in-10-medical-products-in-developinG-countries-is-substandard-or-falsified

[2] S. Ozawa, D. R. Evans, S. Bessias, D. G. Haynie, T. T. Yemeke, S. K. Laing, and J. E. Herrington, "Prevalence and estimated economic burden of substandard and falsified medicines in low- and middle-income countries: A systematic review and meta-analysis," *JAMA Network Open*, vol. 1, no. 4, p. e181662, 2018.

[3] Central Drugs Standard Control Organisation, "NSQ alerts: NSQ drugs and spurious drugs (public viewer, SUGAM portal)," https://cdscoonline.gov.in/CDSCO/viewPublicNSQDrug, 2026, accessed 25 Oct 2026.

[4] ——, "Guidelines on recall and rapid alert system for drugs, version 2024, document no. cdsco/rras ver. 00, effective 18 july 2024; with sop qms-gnl-010," Directorate General of Health Services, Ministry of Health and Family Welfare, Government of India, 2024, accessed 26 Oct 2026. Effective 18/07/2024 and label VERSION: 2024 confirmed in the PDF text. [Online]. Available: https://cdsco.gov.in/opencms/export/sites/CDSCO_WEB/Pdf-documents/biologicals/Guidelinerapid24.pdf

[5] ——, "FAQs on implementation of GSR 823(E) dated 17th november, 2022 with respect to bar code or QR code on top 300 brands," Jul. 2023, 2 pp.; dated 21 Jul 2023 per the file name. Accessed 25 Oct 2026. [Online]. Available: https://cdsco.gov.in/opencms/resources/UploadCDSCOWeb/2018/UploadPublic_NoticesFiles/Final%20FAQs%20on%20QR%20code%2021.07.2023.pdf

[6] World Health Organization, "Policy paper on traceability of medical products," World Health Organization, Geneva, Tech. Rep., Mar. 2021, accessed 25 Oct 2026. [Online]. Available: https://www.who.int/publications/i/item/policy-paper-on-traceability-of-medical-products

[7] Central Drugs Standard Control Organisation, "Notice: Availability of not of standard quality (NSQ) alert on new link in CDSCO website," CDSCO Enforcement and Intelligence Cell, signed by R. S. Raghuvanshi, DCG(I), Aug. 2025, dated 28 Aug 2025; new link https://cdsco.gov.in/opencms/opencms/en/Notifications/nsq-drugs/. Accessed 25 Oct 2026. [Online]. Available: https://cdsco.gov.in/opencms/resources/UploadCDSCOWeb/2018/UploadAlertsFiles/aret%20all%20Stakeholders.pdf

[8] ——, "NSQ alerts," https://cdsco.gov.in/opencms/opencms/en/Notifications/nsq-drugs/, 2026, accessed 25 Oct 2026.

[9] S. Ozawa, H.-H. Chen, Y.-F. A. Lee, C. R. Higgins, and T. T. Yemeke, "Characterizing medicine quality by active pharmaceutical ingredient levels: A systematic review and meta-analysis across low- and middle-income countries," *The American Journal of Tropical Medicine and Hygiene*, vol. 106, no. 6, pp. 1778–1790, 2022.

[10] S. Roy, "Over 14,300 drug samples fail quality tests in five years: Minister," Medical Dialogues (news), Dec. 2025, 18 Dec 2025; reports Lok Sabha reply by Health Minister J. P. Nadda. Accessed 25 Oct 2026. [Online]. Available: https://medicaldialogues.in/news/industry/pharma/over-14300-drug-samples-fail-quality-tests-in-five-years-minister-160877

[11] S. Vickers, M. Bernier, S. Zambrzycki, F. M. Fernandez, P. N. Newton, and C. Caillet, "Field detection devices for screening the quality of medicines: a systematic review," *BMJ Global Health*, vol. 3, no. 4, p. e000725, 2018.

[12] S. Kovacs, S. E. Hawes, S. N. Maley, E. Mosites, L. Wong, and A. Stergachis, "Technologies for detecting falsified and substandard drugs in low and middle-income countries," *PLoS ONE*, vol. 9, no. 3, p. e90601, 2014.

[13] A. Ciapponi, M. Donato, A. M. Gülmezoglu, T. Alconada, and A. Bardach, "Mobile apps for detecting falsified and substandard drugs: A systematic review," *PLOS ONE*, vol. 16, no. 2, p. e0246061, 2021.

[14] Blacklisted, "Blacklisted: record of indian regulatory action on medicines, food, cosmetics and AYUSH," 2026, website, last updated 10 Oct 2026. Accessed 25 Oct 2026. [Online]. Available: https://blacklisted.co.in

[15] P. Rastogi, "Nazar," 2026, gitHub repository, MIT licence; README states live 18 Oct 2026. Accessed 25 Oct 2026. [Online]. Available: https://github.com/pranshurastogi/Nazar

[16] shaurya703, "SafeDose," 2026, gitHub repository. Accessed 25 Oct 2026. [Online]. Available: https://github.com/shaurya703/safedose

[17] ibnweb3, "RecallOps India," 2026, gitHub repository, neatHack 2026 project; code MIT. Accessed 25 Oct 2026. [Online]. Available: https://github.com/ibnweb3/recallops-india

[18] J. Koponen, K. Haataja, and P. Toivanen, "Novel deep learning application: Recognizing inconsistent characters on pharmaceutical packaging," *F1000Research*, vol. 12, p. 427, 2024, version 2; peer review: 1 approved with reservations, 1 not approved.

[19] Y. Du, C. Li, R. Guo, X. Yin, W. Liu, J. Zhou, Y. Bai, Z. Yu, Y. Yang, Q. Dang, and H. Wang, "PP-OCR: A practical ultra lightweight OCR system," 2020.

[20] C. Li, W. Liu, R. Guo, X. Yin, K. Jiang, Y. Du, Y. Du, L. Zhu, B. Lai, X. Hu, D. Yu, and Y. Ma, "PP-OCRv3: More attempts for the improvement of ultra lightweight OCR system," 2022.

[21] RapidAI Team, "Rapid OCR: OCR toolbox," GitHub repository, 2021, accessed 25 Oct 2026. [Online]. Available: https://github.com/RapidAI/RapidOCR

[22] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 2017, pp. 1321–1330. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html

[23] Z. He, C. Zhang, Z. Wu, Z. Chen, Y. Zhan, Y. Li, Z. Zhang, X. Wang, and M. Qiu, "Seeing is believing? mitigating OCR hallucinations in multimodal large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, arXiv:2506.20168; accepted by NeurIPS 2025 per arXiv comment; proceedings pages not available. [Online]. Available: https://arxiv.org/abs/2506.20168

[24] R. A. Tavares, "Comparison of image preprocessing techniques for vehicle license plate recognition using OCR: Performance and accuracy evaluation," 2024.

[25] M. D. Kernighan, K. W. Church, and W. A. Gale, "A spelling correction program based on a noisy channel model," in *COLING 1990 Volume 2: Papers presented to the 13th International Conference on Computational Linguistics*, Helsinki, Finland, 1990, pp. 205–210. [Online]. Available: https://aclanthology.org/C90-2036/

[26] E. Brill and R. C. Moore, "An improved error model for noisy channel spelling correction," in *Proceedings of the 38th Annual Meeting of the Association for Computational Linguistics*. Hong Kong: Association for Computational Linguistics, Oct. 2000, pp. 286–293. [Online]. Available: https://aclanthology.org/P00-1037/

[27] O. Kolak and P. Resnik, "OCR error correction using a noisy channel model," in *Proceedings of the Second International Conference on Human Language Technology Research (HLT 2002)*, 2002, pp. 257–262.

[28] L. von Ahn, B. Maurer, C. McMillen, D. Abraham, and M. Blum, "reCAPTCHA: Human-based character recognition via web security measures," *Science*, vol. 321, no. 5895, pp. 1465–1468, Sep. 2008.

[29] R. Laroca, L. A. Zanlorensi, G. R. Gonçalves, E. Todt, W. R. Schwartz, and D. Menotti, "An efficient and layout-independent automatic license plate recognition system based on the YOLO detector," *IET Intelligent Transport Systems*, vol. 15, no. 4, pp. 483–503, 2021.

[30] Seattle Office of Inspector General, "Surveillance technology usage review: Automated license plate reader – patrol (2021 and 2022)," City of Seattle, Office of Inspector General, Surveillance Technology Usage Review, as required by Seattle Municipal Code 14.18.060, Sep. 2023, published 29 Sep 2023. Accessed 25 Oct 2026. [Online]. Available: https://www.seattle.gov/documents/departments/oig/audits/surveillancetechnologyusagereview-automatedlicenseplatereader-patrol(2021and2022).pdf

[31] United States Court of Appeals for the Ninth Circuit, "Green v. city and county of san francisco, no. 11-17892 (opinion filed 12 may 2014)," 751 F.3d 1039 (9th Cir. 2014), 2014, reporter citation confirmed via CourtListener. Accessed 25 Oct 2026. [Online]. Available: https://cdn.ca9.uscourts.gov/datastore/opinions/2014/05/12/11-17892.pdf

[32] I. P. Fellegi and A. B. Sunter, "A theory for record linkage," *Journal of the American Statistical Association*, vol. 64, no. 328, pp. 1183–1210, 1969.

[33] W. E. Winkler, "Overview of record linkage and current research directions," U.S. Census Bureau, Statistical Research Division, Washington, DC, Research Report Series Statistics #2006-2, 2006. [Online]. Available: https://www.census.gov/content/dam/Census/library/working-papers/2006/adrm/rrs2006-02.pdf

[34] P. Christen, "A survey of indexing techniques for scalable record linkage and deduplication," *IEEE Transactions on Knowledge and Data Engineering*, vol. 24, no. 9, pp. 1537–1555, 2012.

[35] G. Papadakis, D. Skoutas, E. Thanos, and T. Palpanas, "Blocking and filtering techniques for entity resolution: A survey," *ACM Computing Surveys*, vol. 53, no. 2, pp. 1–42, 2020, published online 20 Mar 2020; preprint arXiv:1905.06167.

[36] R. Peeters, A. Steiner, and C. Bizer, "Entity matching using large language models," in *Proceedings of the 28th International Conference on Extending Database Technology (EDBT)*. OpenProceedings.org, 2025, pp. 529–541, arXiv:2310.11244.

[37] M. Neelakantan, P. Sharma, and A. Kulkarni, "Look-alike, sound-alike (LASA) drugs in India," *The Lancet Regional Health - Southeast Asia*, vol. 26, p. 100425, 2024.

[38] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.03629; venue per arXiv v3 comments. [Online]. Available: https://arxiv.org/abs/2210.03629

[39] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, pp. 68 539–68 551. [Online]. Available: https://arxiv.org/abs/2302.04761

[40] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, pp. 8634–8652. [Online]. Available: https://arxiv.org/abs/2303.11366

[41] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, pp. 46 534–46 594. [Online]. Available: https://arxiv.org/abs/2303.17651

[42] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2310.01798. [Online]. Available: https://arxiv.org/abs/2310.01798

[43] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. Saldyt, and A. Murthy, "LLMs can't plan, but can help planning in LLM-Modulo frameworks," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, Vienna, Austria, 2024, arXiv:2402.01817; volume and venue per arXiv listing, PMLR page not opened. [Online]. Available: https://arxiv.org/abs/2402.01817

[44] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," 2024, arXiv preprint, 17 Jun 2024. Accessed 25 Oct 2026. [Online]. Available: https://arxiv.org/abs/2406.12045

[45] E. S. and B. Zhang, "Building effective agents," Anthropic Engineering blog, published 19 Dec 2024, 2024, accessed 25 Oct 2026. First author's surname shown only as "Erik S." on the opened page. [Online]. Available: https://www.anthropic.com/engineering/building-effective-agents

[46] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan, "AI agents that matter," 2024, arXiv preprint, 1 Jul 2024. Accessed 25 Oct 2026. [Online]. Available: https://arxiv.org/abs/2407.01502

[47] P. Belcak, G. Heinrich, S. Diao, Y. Fu, X. Dong, S. Muralidharan, Y. C. Lin, and P. Molchanov, "Small language models are the future of agentic AI," arXiv preprint arXiv:2506.02153, 2025, v1 2 Jun 2025; v3 22 Sep 2026; accessed 25 Oct 2026. [Online]. Available: https://arxiv.org/abs/2506.02153

[48] C. K. Chow, "On optimum recognition error and reject tradeoff," *IEEE Transactions on Information Theory*, vol. 16, no. 1, pp. 41–46, 1970.

[49] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017, arXiv:1705.08500. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html

[50] A. Cocchieri, L. Ragazzi, G. Tagliavini, and G. Moro, "LLMs (almost) never abstain under medical uncertainty," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. San Diego, California, United States: Association for Computational Linguistics, 2026, pp. 29 573–29 613.

[51] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Human Factors*, vol. 52, no. 3, pp. 381–410, 2010.

[52] Z. Buçinca, M. B. Malaya, and K. Z. Gajos, "To trust or to think: Cognitive forcing functions can reduce overreliance on AI in AI-assisted decision-making," *Proceedings of the ACM on Human-Computer Interaction*, vol. 5, no. CSCW1, pp. 1–21, 2021.

[53] E. Batteux, S. Bonfield, L. F. Jones, H. Carter, N. Gold, R. Amlot, T. Marteau, and D. Weston, "Impact of residual risk messaging to reduce false reassurance following test-negative results from asymptomatic coronavirus (SARS-CoV-2) testing: an online experimental study of a hypothetical test," *BMJ Open*, vol. 12, no. 3, p. e056533, 2022.

[54] National Highway Traffic Safety Administration, "49 CFR 573.15: Public availability of motor vehicle recall information," Electronic Code of Federal Regulations, 2026, text version dated 1 Oct 2026, read via the eCFR API 26 Oct 2026; section number and title confirmed. [Online]. Available: https://www.ecfr.gov/current/title-49/subtitle-B/chapter-V/part-573/section-573.15

[55] Ollama, "Structured outputs," Ollama documentation, 2026, accessed 25 Oct 2026. [Online]. Available: https://docs.ollama.com/capabilities/structured-outputs

[56] OpenTelemetry Authors, "Semantic conventions for generative ai (semantic-conventions-genai repository)," 2026, accessed 25 Oct 2026. The opentelemetry.io page at /docs/specs/semconv/gen-ai/ only redirects here. [Online]. Available: https://github.com/open-telemetry/semantic-conventions-genai

[57] Arize AI, "OpenInference: conventions and plugins for tracing AI applications," 2026, accessed 25 Oct 2026. [Online]. Available: https://github.com/Arize-ai/openinference

[58] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[59] L. D. Brown, T. T. Cai, and A. DasGupta, "Interval estimation for a binomial proportion," *Statistical Science*, vol. 16, no. 2, pp. 101–133, 2001.

[60] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[61] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.

[62] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proceedings of the National Academy of Sciences*, vol. 115, no. 11, pp. 2600–2606, 2018.